Automaton Auditor — Final Architecture Report

Automaton Auditor · Final Submission

Table of Contents

1. Project Overview
2. Architecture Decisions
    - 2.1 Why Pydantic + TypedDict over Plain Dicts
    - 2.2 State Reducers: Preventing Parallel Overwrites
    - 2.3 AST Parsing Strategy
    - 2.4 Sandboxing Strategy for Repository Cloning
    - 2.5 PDF Ingestion: RAG-Lite Without a Vector Store
    - 2.6 Vision Analysis: Graceful Degradation
    - 2.7 LLM Provider Selection for the Judicial Layer
    - 2.8 Deterministic Chief Justice: No Fourth LLM Call
3. Final StateGraph Flow
    - 3.1 Complete Node Topology
    - 3.2 Conditional Edge Logic
    - 3.3 Full State Data Flow
4. Rubric Dimension Coverage
5. Judicial Layer Implementation
    - 5.1 Judge Personas and Structured Output
    - 5.2 Chief Justice Conflict-Resolution Rules
    - 5.3 Variance Re-evaluation and Dissent
6. Implementation Status
7. Environment and Observability

---

1. Project Overview

The **Automaton Auditor** is a hierarchical multi-agent system built on LangGraph. It accepts a GitHub repository URL and a PDF report as inputs and produces a structured forensic audit across rubric dimensions — verifying code artifacts, architectural patterns, documentation quality, and visual diagrams.

The system is structured as a **Digital Courtroom** with three layers:

- **Detective Layer** — `RepoInvestigator`, `DocAnalyst`, and `VisionInspector` run in parallel, each collecting objective `Evidence` objects tied to specific rubric dimensions.
- **Judicial Layer** — `Prosecutor`, `Defense`, and `Tech Lead` judge nodes run in parallel after evidence aggregation. Each applies a distinct adversarial lens to produce a `JudicialOpinion` per dimension.
- **Supreme Court** — `ChiefJustice` applies deterministic Python conflict-resolution rules to synthesise a final `AuditReport`.

All three layers are fully implemented and wired in the final submission.

---

2. Architecture Decisions

2.1 Why Pydantic + TypedDict over Plain Dicts

The state management layer uses a hybrid of `TypedDict` (for the graph-level `AgentState`) and Pydantic `BaseModel` (for nested value objects like `Evidence`, `JudicialOpinion`, `CriterionResult`, and `AuditReport`).

The problem with plain dicts:

```
# Fragile: no schema, no validation, silent bugs
state = {"evidences": {"repo": [{"found": True, "confidence": "high"}]}}
```

- Keys can be misspelled silently.
- Values have no type constraints — `confidence: "high"` instead of `0.9` is accepted at runtime.
- Parallel nodes writing to the same key will overwrite each other's data with no indication of failure.
- No `.model_dump()` serialisation for output JSON.

The chosen approach:

```
class Evidence(BaseModel):
    goal: str
    found: bool
    content: Optional[str]
    location: str
    rationale: str
    confidence: float = Field(ge=0.0, le=1.0)
```

Every detective node constructs `Evidence` objects. Pydantic enforces the `ge=0.0, le=1.0` range on `confidence` at instantiation time — not at test time. `JudicialOpinion.score` is similarly constrained to `ge=1, le=5`, making it impossible for a judge to emit a score of `0` or `6` without raising a `ValidationError`.

`AgentState` is a `TypedDict` rather than a Pydantic model because LangGraph's `StateGraph` requires a `TypedDict`-compatible schema for its state channel system.

Type hierarchy:

```
classDiagram
    class AgentState {
        +str repo_url
        +str pdf_path
        +List~Dict~ rubric_dimensions
        +Dict~str, List~Evidence~~ evidences
        +List~JudicialOpinion~ opinions
        +Optional~AuditReport~ final_report
    }

    class Evidence {
        +str goal
        +bool found
        +Optional~str~ content
        +str location
        +str rationale
        +float confidence
    }

    class JudicialOpinion {
        +Literal judge
        +str criterion_id
        +int score
        +str argument
        +List~str~ cited_evidence
    }

    class CriterionResult {
        +str dimension_id
        +str dimension_name
        +int final_score
        +List~JudicialOpinion~ judge_opinions
        +Optional~str~ dissent_summary
        +str remediation
```

```
}

class AuditReport {
    +str repo_url
    +str executive_summary
    +float overall_score
    +List~CriterionResult~ criteria
    +str remediation_plan
}

AgentState "1" --> "many" Evidence : evidences dict
AgentState "1" --> "many" JudicialOpinion : opinions list
AgentState "1" --> "1" AuditReport : final_report
CriterionResult "1" --> "3" JudicialOpinion : judge_opinions
AuditReport "1" --> "many" CriterionResult : criteria
```

2.2 State Reducers: Preventing Parallel Overwrites

When three detective nodes run in parallel, each writes into `state["evidences"]`. Without a reducer, the last node to finish silently clobbers the other two nodes' results — a race condition that produces no error and no warning. The same problem occurs with the three judge nodes writing into `state["opinions"]`.

LangGraph resolves this with `Annotated` type hints on the state fields:

```
class AgentState(TypedDict):
    evidences: Annotated[Dict[str, List[Evidence]], operator.ior]
    opinions: Annotated[List[JudicialOpinion], operator.add]
```

- `operator.ior` (`|=`) performs a **dict merge**: each parallel detective writes into a separate key (`"repo"`, `"doc"`, `"vision"`), and the reducer combines all three dicts without any key colliding.
- `operator.add` performs **list concatenation**: judge opinions from three parallel judge nodes are concatenated into a single list, preserving all opinions for the Chief Justice.

Without reducer (race condition):

```
sequenceDiagram
    participant R as RepoInvestigator
    participant D as DocAnalyst
    participant V as VisionInspector
    participant S as AgentState
```

```
    R->>S: state["evidences"] = {"repo": [...]}
    D->>S: state["evidences"] = {"doc": [...]}
    note over S: ⚠️ "repo" key is overwritten
    V->>S: state["evidences"] = {"vision": [...]}
    note over S: ⚠️ "doc" key is overwritten
```

With `operator.ior` reducer (correct merge):

```
sequenceDiagram
    participant R as RepoInvestigator
    participant D as DocAnalyst
    participant V as VisionInspector
    participant S as AgentState

    R->>S: {"repo": [...]}
    D->>S: {"doc": [...]}
    V->>S: {"vision": [...]}
    note over S: reducer merges all three dicts
    note over S: evidences = {"repo": [...], "doc": [...], "vision": [...]}
```

2.3 AST Parsing Strategy

The `analyze_graph_structure` function in `src/tools/repo_tools.py` uses Python's built-in `ast` module rather than regular expressions.

Why not regex?

Regex on source code is brittle. Consider these equivalent constructions:

```
# Pattern A
builder.add_edge("entry", "repo_investigator")

# Pattern B
g = builder
g.add_edge(
    "entry",
    "repo_investigator"
)

# Pattern C — regex would miss this entirely
edges = [("entry", "repo_investigator")]
for src, dst in edges:
    builder.add_edge(src, dst)
```

A regex pattern for `add_edge\("(\w+)",\s*"(\w+)"\)` matches Pattern A but misses B and C. The AST visitor handles A and B correctly (Pattern C is intentionally out of scope as it requires data-flow analysis).

AST visitor design:

```
class _GraphVisitor(ast.NodeVisitor):
    def visit_Call(self, node: ast.Call):
        func = node.func
        if isinstance(func, ast.Attribute):
            if func.attr == "add_node":       # → collect node name
            if func.attr == "add_edge":       # → collect (src, dst) tuple
            if func.attr == "add_conditional_edges":  # → increment counter
        self.generic_visit(node)
```

The visitor walks the full AST, so it catches nested calls, calls in helper functions, and calls across multiple builder variable names.

Fan-out and fan-in detection:

```
src_counts = Counter(src for src, _ in visitor.edges)
dst_counts = Counter(dst for _, dst in visitor.edges)
has_parallel = any(v >= 2 for v in src_counts.values())   # one src → many dst
has_fan_in   = any(v >= 2 for v in dst_counts.values())   # many src → one dst
```

Why not `libcst`, `tree-sitter`, or live introspection?

Alternative	Rejection reason
`libcst`	Preserves whitespace and comments, but adds a third-party dependency and a much larger API surface. The auditor only needs method call names and their string literal arguments — a use case squarely within `ast`'s sweet spot.
`tree-sitter`	Language-agnostic and faster on multi-MB files, but requires compiled binary extensions per platform. The auditor's target files are small Python modules; the performance gain does not justify a native extension dependency.

`importlib` / live introspection	Importing the target module would **execute the module's top-level code**, including any `StateGraph.compile()` calls. Executing untrusted code from an unknown repository is the exact threat model the sandboxing strategy (§2.4) was designed to avoid.

2.4 Sandboxing Strategy for Repository Cloning

Cloning unknown repositories is a high-risk operation. The `clone_repo_sandboxed` function in `src/tools/repo_tools.py` applies four layers of protection:

```
flowchart TD
  A[Receive repo_url] --> B{URL validation}
  B -->|not https:// or git@| C[Return CloneResult ok=False\nerror=invalid_url]
  B -->|valid prefix| D[tempfile.mkdtemp\ncreates isolated temp dir]
  D --> E["subprocess.run(\n  ['git','clone', repo_url, tmp_dir],\n  capture_output=True\n)"]
  E --> F{return code == 0?}
  F -->|no| G[Return CloneResult ok=False\nerror=clone_failed\ndetails=stderr]
  F -->|yes| H[Return CloneResult ok=True\ncloned_path=tmp_dir]
```

Layer 1 — URL allow-listing: Only `https://` and `git@` prefixes are accepted. Any other value returns an error before any subprocess is spawned. This prevents injection strings like `; rm -rf /` embedded in a URL.

Layer 2 — Temp directory isolation: `tempfile.mkdtemp()` creates a directory with a randomised name under the OS's temp path. The cloned repository never touches the working directory of the auditor process.

Layer 3 — `subprocess.run` with an explicit argument list (no `shell=True`): When `shell=True` is used, the command string is passed to `/bin/sh -c`, which means shell metacharacters in `repo_url` are interpreted. An explicit argument list bypasses the shell entirely:

```
# Vulnerable — shell interprets metacharacters in repo_url
subprocess.run(f"git clone {repo_url} {tmp}", shell=True)

# Safe — repo_url is passed as a literal argument to git
subprocess.run(["git", "clone", repo_url, tmp], capture_output=True)
```

Layer 4 — Return-code checking and structured errors: `returncode != 0` is always checked. `stderr` is captured and returned inside `CloneResult.details`. No exception propagates to the graph.

2.5 PDF Ingestion: RAG-Lite Without a Vector Store

The `ingest_pdf` function uses PyMuPDF (`fitz`) to extract text page-by-page, then splits it into overlapping character-window chunks (~1800 characters, ~75% overlap between consecutive chunks).

Why chunking instead of full-document context?

LLM context windows are finite. A 30-page PDF with diagrams can exceed 40,000 tokens. Chunking allows targeted retrieval: instead of passing the entire PDF to an LLM, the `query_pdf` function uses a TF-score (term frequency) ranking to surface the top-k most relevant chunks for a given query.

TF scoring (no external vector DB required):

```
def _tf_score(query_tokens, doc_tokens):
    score = sum(doc_tokens.count(qt) / len(doc_tokens) for qt in query_tokens)
    return score * math.log(1 + len(doc_tokens))
```

The log factor discounts extremely long chunks that would otherwise always win by sheer volume.

Why PyMuPDF (`fitz`) over other PDF parsers?

Library	Verdict
`pdfplumber`	Excellent for table extraction, but 3–5× slower than PyMuPDF for raw text extraction. Overkill for sequential plain text.
`pdfminer.six`	Low-level and configurable, but requires manual layout analysis object management. Additional complexity not justified.

`pypdf`	Lightweight and zero C-extension, but loses text ordering on multi-column PDFs with embedded fonts.
`PyMuPDF` (`fitz`)	Native C extension (libmupdf), fastest raw text extraction in benchmarks, robust on embedded fonts, and provides `page.get_images()` for diagram PNGs. A single library for both text (DocAnalyst) and images (VisionInspector) avoids loading the file twice.

2.6 Vision Analysis: Graceful Degradation

`VisionInspector` is designed so that the absence of a vision-capable API key never crashes the graph. The execution path degrades gracefully at each step:

```
flowchart TD
  A[pdf_path] --> B[extract_images_from_pdf]
  B --> C{images found?}
  C -->|no| D["Evidence(found=False,\nrationale='No images in PDF')"]
  C -->|yes| E{GOOGLE_API_KEY set?}
  E -->|yes| F[Gemini 2.5 Flash\nmultimodal call]
  E -->|no| G{ANTHROPIC_API_KEY set?}
  G -->|yes| H[Claude 3.5 Haiku\nmultimodal call]
  G -->|no| I["Evidence(found=False,\nrationale='No vision model configured')"]
  F --> J{parse JSON response}
  H --> J
  J -->|success| K["Evidence(found=True,\nclassification=...,\nconfidence=...)"]
  J -->|exception| L["Evidence(found=False,\nerror=api_error)"]
```

The five classification labels are aligned with the rubric's `swarm_visual` dimension:

Label	Meaning
`accurate_stategraph`	Shows parallel fan-out/fan-in LangGraph nodes

`sequence_diagram`	UML sequence / step arrows
`generic_flowchart`	Flowchart without parallelism
`linear_pipeline`	Strictly sequential, no parallelism
`other`	Unclassifiable

2.7 LLM Provider Selection for the Judicial Layer

The Judicial Layer uses LLMs to generate structured `JudicialOpinion` objects. Provider selection was driven by **access constraints and task-fit** at the time of project undertaking, not abstract benchmark comparisons.

Primary constraint: existing subscription access

An active Anthropic subscription was already in place at project start. This made Claude the zero-friction choice for all text reasoning tasks — no new billing setup, no quota approval wait, and predictable pricing already understood from prior use.

Model tier: Haiku for judges

The three judge personas (`Prosecutor`, `Defense`, `Tech Lead`) are implemented against `ChatAnthropic` with `claude-haiku-4-5-20251001`. Haiku was chosen over Sonnet after weighing cost against the structured output constraint: the `JudicialOpinion` schema requires a `cited_evidence: List[str]` field with actual dimension IDs. Haiku's structured output adherence was sufficient for this bounded task — it receives explicit instructions listing all required fields, and the retry loop (up to 3 attempts) catches any validation failures before falling back to a score-1 opinion.

API key dependency summary:

Use case	Primary	Fallback	Behaviour with no key
Judge personas (all three)	`ANTHROPIC_API_KEY` (Claude Haiku 4-5)	None	Node emits fallback `JudicialOpinion(score=1, argument="Opinion emission failed...")`
Vision diagram analysis	`GOOGLE_API_KEY` (Gemini 2.5 Flash)	`ANTHROPIC_API_KEY` (Claude Haiku)	`Evidence(found=False, rationale="No vision model configured")`

2.8 Deterministic Chief Justice: No Fourth LLM Call

The obvious synthesis alternative — passing all three `JudicialOpinion` objects to an LLM and asking it to produce a final score — was rejected for three reasons:

1. **Non-determinism.** Two consecutive runs on identical evidence can yield different final scores when synthesis is LLM-driven. An auditing system that produces different verdicts on the same input is not auditable. The hardcoded Python rules produce the same `CriterionResult` for the same inputs every time.
2. **Accountability.** If a score is disputed, "the LLM decided" is not an answer. The deterministic rules (`Security Override`, `Fact Supremacy`, `Variance Re-evaluation`, `Functionality Weight`) are named, version-controlled, and readable directly. The audit trail is the code itself.
3. **Cost and latency.** The Judicial Layer already makes LLM calls in parallel for three judges across all rubric dimensions. Adding a fourth synthesis call per dimension would roughly double the token spend with no accuracy benefit — conflict resolution is precedence application, not creative reasoning.

The LLM is reserved for tasks that require language understanding (interpreting evidence, constructing arguments). Arithmetic precedence rules belong in Python.

---

3. Final StateGraph Flow

3.1 Complete Node Topology

The final graph implements all three layers: Detective, Judicial, and Supreme Court (Chief Justice).

```
flowchart TD
   START([START]) --> entry_node

   entry_node -->|valid inputs| repo_investigator
   entry_node -->|valid inputs| doc_analyst
   entry_node -->|valid inputs| vision_inspector
   entry_node -->|missing / invalid inputs| abort

   subgraph "Detective Layer — parallel fan-out"

repo_investigator["RepoInvestigator\n──────────────────\ngit_forensic_analysis\nstate_mana
gement_rigor\ngraph_orchestration\nsafe_tool_engineering"]
      doc_analyst["DocAnalyst\n──────────────────\ntheoretical_depth\nreport_accuracy"]
      vision_inspector["VisionInspector\n──────────────────\nswarm_visual\n(degrades to
found=False if no key)"]
   end

   repo_investigator --> evidence_aggregator
   doc_analyst --> evidence_aggregator
   vision_inspector --> evidence_aggregator

   evidence_aggregator["── EvidenceAggregator ──\noperator.ior dict merge\nbackfills
missing dimensions → found=False"]

   abort["abort_node\nemits found=False\nfor all dimensions"]
   abort --> END([END])

   evidence_aggregator --> prosecutor
   evidence_aggregator --> defense
   evidence_aggregator --> tech_lead

   subgraph "Judicial Layer — parallel fan-out"
      prosecutor["Prosecutor\nadversarial lens · score 1–5\n(parse error → fallback score=1)"]
      defense["Defense\noptimistic lens · score 1–5\n(parse error → fallback score=1)"]
      tech_lead["Tech Lead\npragmatic lens · scores 1, 3, or 5 only\n(parse error → fallback
score=1)"]
   end

   prosecutor --> chief_justice
```

```
    defense --> chief_justice
    tech_lead --> chief_justice

    chief_justice["ChiefJustice\n── deterministic rules ──\nSecurity Override · Fact Supremacy\nVariance Re-eval · Functionality Weight"]

    chief_justice --> END
```

3.2 Conditional Edge Logic

The `entry_node` validates inputs before the detective fan-out:

```
flowchart LR
    entry_node -->|_input_router| router{router}
    router -->|"repo_url missing\nor pdf_path missing\nor invalid URL prefix"| abort
    router -->|"valid https:// or git@ URL\nand pdf_path provided"| repo_investigator
    entry_node -.->|direct edge| doc_analyst
    entry_node -.->|direct edge| vision_inspector
```

`doc_analyst` and `vision_inspector` receive direct (non-conditional) edges from `entry_node` because they are independent of URL validity — they only require `pdf_path`. The conditional router gates only `RepoInvestigator`.

3.3 Full State Data Flow

```
sequenceDiagram
    participant User
    participant Entry as entry_node
    participant Repo as RepoInvestigator
    participant Doc as DocAnalyst
    participant Vision as VisionInspector
    participant Agg as EvidenceAggregator
    participant P as Prosecutor
    participant D as Defense
    participant TL as TechLead
    participant CJ as ChiefJustice

    User->>Entry: AgentState{repo_url, pdf_path, rubric_dimensions}
    Entry->>Entry: validate inputs, init evidence buckets

    par Detective Fan-Out
        Entry->>Repo: AgentState (parallel)
        Entry->>Doc: AgentState (parallel)
        Entry->>Vision: AgentState (parallel)
    end
```

```
Repo-->>Agg: {"repo": [Evidence×4]}
Doc-->>Agg: {"doc": [Evidence×2]}
Vision-->>Agg: {"vision": [Evidence×1]}
note over Agg: operator.ior merges all three dicts

par Judicial Fan-Out
   Agg->>P: AgentState (parallel)
   Agg->>D: AgentState (parallel)
   Agg->>TL: AgentState (parallel)
end

P-->>CJ: [JudicialOpinion×N]
D-->>CJ: [JudicialOpinion×N]
TL-->>CJ: [JudicialOpinion×N]
note over CJ: operator.add concatenates all opinions

CJ->>CJ: Apply conflict-resolution rules per dimension
CJ-->>User: AgentState{final_report: AuditReport}
```

---

4. Rubric Dimension Coverage

The table below maps each rubric dimension to the detective node that collects evidence, and to the judge personas that evaluate it:

Dimension	Detective	Tool Function	Confidence Heuristic
`git_forensic_analysis`	RepoInvestigator	`extract_git_history`	0.85 if ≥2 phases detected in commit messages; 0.4 otherwise
`state_management_rigor`	RepoInvestigator	AST scan of `src/state.py`	0.9 if Pydantic + TypedDict + reducers all

			present; 0.5 otherwise
`graph_orchestration`	RepoInvestigator	`analyze_graph_structure`	0.9 if fan-out AND fan-in detected; 0.5 otherwise
`safe_tool_engineering`	RepoInvestigator	`clone_repo_sandboxed` (self-audit)	0.95 (constant — deterministic check)
`theoretical_depth`	DocAnalyst	`query_pdf` × 6 rubric queries	`matched_count / total_queries`
`report_accuracy`	DocAnalyst	`extract_file_paths_from_text` + path existence check	`verified_count / mentioned_count`
`swarm_visual`	VisionInspector	`extract_images_from_pdf` + `analyze_diagram`	LLM-reported confidence × 0.5 if not `accurate_state graph`

All seven dimensions are evaluated by all three judge personas (`Prosecutor`, `Defense`, `Tech Lead`) and synthesised by `ChiefJustice` into a single `CriterionResult` per dimension.

---

5. Judicial Layer Implementation

5.1 Judge Personas and Structured Output

Three judge node functions run in parallel after `EvidenceAggregator`: `prosecutor_node`, `defense_node`, and `tech_lead_node`. Each is a LangGraph node function implemented in `src/nodes/judges.py`.

All three share the same `_judge_node` factory, which:

1. Iterates over every rubric dimension in `state["rubric_dimensions"]`.
2. Looks up the corresponding `Evidence` object from `state["evidences"]` using `ev.goal == dim_id`.
3. Calls the judge LLM bound to `JudicialOpinion` via `.with_structured_output(JudicialOpinion)`.
4. Retries up to `MAX_RETRIES=3` on `ValidationError` or any exception.
5. Falls back to `JudicialOpinion(score=1, argument="Opinion emission failed...")` on total failure.

Persona mandates:

Persona	Core mandate	Scoring bias
Prosecutor	"Trust No One. Assume Vibe Coding."	Score 1–2 for violations; penalises gaps, security flaws, laziness
Defense	"Reward Effort and Intent."	Score 3–5; credits iteration, architectural understanding, creative workarounds
Tech Lead	"Evaluate architectural soundness only."	Scores **1, 3, or 5 only** (never 2 or 4); acts as tie-breaker

The Tech Lead's restriction to odd scores (1, 3, 5) is an explicit scoring anchor that prevents hedging — the persona must commit to a clear verdict rather than splitting the difference between Prosecutor and Defense.

Structured output binding:

```
llm = ChatAnthropic(model=_MODEL, temperature=0)
judge_llm = llm.with_structured_output(JudicialOpinion)
```

`temperature=0` is set on all judge chains to maximise schema adherence and reduce the probability of freeform text responses that would trigger retries.

Parallel accumulation via `operator.add`:

Because `AgentState.opinions` is annotated with `operator.add`, the three judge nodes each return `{"opinions": [JudicialOpinion, ...]}` and LangGraph concatenates all three lists before passing them to `chief_justice`. No opinion is overwritten.

5.2 Chief Justice Conflict-Resolution Rules

The `ChiefJusticeNode` in `src/nodes/justice.py` is a pure Python function — no LLM call. It groups all opinions by `criterion_id` and applies five named rules in priority order:

```
flowchart TD
  A[Receive JudicialOpinions\nfor one criterion] --> B{Prosecutor argument\ncontains security signal?}
  B -->|yes| C[Security Override\nCap final score at 3]
  B -->|no| D{Detective found=False\nAND Defense score >= 4?}
  D -->|yes| E[Fact Supremacy\nDemote Defense score\nby 2; take minimum with TechLead]
  D -->|no| F{Dimension is\narchitecture criterion?}
  F -->|yes| G[Functionality Weight\nTechLead weight × 2\nin weighted average]
  F -->|no| H{Score variance\nacross 3 judges > 2?}
  H -->|yes| I[Variance Re-evaluation\nTechLead weight × 2\nas tie-breaker]
  H -->|no| J[Default Weighted Average\nProsecutor×1 + Defense×1\n+ TechLead×1]
  C --> K[CriterionResult\nwith optional dissent_summary]
  E --> K
  G --> K
  I --> K
  J --> K
```

Rule 1 — Security Override: The Prosecutor's argument is scanned for keyword signals (`"os.system"`, `"security negligence"`, `"shell injection"`, `"unsanitized"`, `"no sandbox"`). When a confirmed security flaw is detected, the final score is capped at 3 regardless of other judges' positions.

Rule 2 — Fact Supremacy: When the detective found the artifact missing (`found=False`) but the Defense assigned score ≥ 4 (excessive generosity), the Defense score is demoted by 2 and the final score is computed as the minimum of the demoted Defense score and the Tech Lead score — grounded in what actually exists in the repository.

Rule 3 — Functionality Weight: For architecture-critical dimensions (`graph_orchestration`, `state_management_rigor`, `safe_tool_engineering`,

`chief_justice_synthesis`), the Tech Lead's score carries double weight in the weighted average. The Tech Lead is the most qualified judge for correctness questions.

Rule 4 — Variance Re-evaluation: When `max(scores) - min(scores) > 2` across the three judges, the panel is significantly divided. The Tech Lead's weight is doubled to break the deadlock with a pragmatic anchor, before the mandatory dissent summary is generated.

Rule 5 — Default weighted average: All three judges contribute equal weight. `round()` is used for integer conversion (Python's banker's rounding).

5.3 Variance Re-evaluation and Dissent

Any criterion where score variance exceeds 2 receives a mandatory `dissent_summary` field in `CriterionResult`. The summary identifies which judge argued for which score, and documents which side the Tech Lead's tie-breaker weight resolved toward.

Weighted score computation:

```
def _weighted_score(prosecutor, defense, tech_lead, tl_weight=1) -> int:
    total_weight = 0
    total_score = 0

    if prosecutor:
        total_score += prosecutor.score * 1
        total_weight += 1
    if defense:
        total_score += defense.score * 1
        total_weight += 1
    if tech_lead:
        total_score += tech_lead.score * tl_weight
        total_weight += tl_weight

    raw = round(total_score / total_weight)
    return max(1, min(5, raw))
```

The `tl_weight` parameter is set to `2` for architecture criteria (Rule 3) and high-variance cases (Rule 4), and to `1` for the default path.

---

6. Implementation Status

All components described in the interim submission's "Known Gaps" section are now implemented.

Component	File	Status
`JudicialOpinion` schema with `security_flagged` semantics	`src/state.py`	✓ Implemented — security signalling via keyword scan in `_has_security_flaw()`
`prosecutor_node` (adversarial judge)	`src/nodes/judges.py`	✓ Implemented — parallel LangGraph node with structured output and retry
`defense_node` (optimistic judge)	`src/nodes/judges.py`	✓ Implemented — parallel LangGraph node with structured output and retry
`tech_lead_node` (pragmatic tie-breaker, scores 1/3/5 only)	`src/nodes/judges.py`	✓ Implemented — parallel LangGraph node with structured output and retry
`chief_justice_node` (deterministic Python synthesis)	`src/nodes/justice.py`	✓ Implemented — five named rules, no LLM call
`audit_report_to_markdown()` serialiser	`src/utils/report_writer.py`	✓ Implemented — structured Markdown output from `AuditReport`
Full graph wiring (detective + judicial fan-out/fan-in)	`src/graph.py`	✓ Implemented — `evidence_aggregator → [prosecutor, defense, tech_lead] → chief_justice → END`

Remaining known limitations:

- **Dynamic rubric routing in judges:** All judge nodes currently iterate over `state["rubric_dimensions"]` and produce one opinion per dimension. A future enhancement would match each dimension to its canonical detective bucket before routing, giving judges explicit context about whether the evidence came from a repo scan, PDF analysis, or vision analysis.
- **Incremental evidence persistence:** Evidence is written only at run completion. If the process crashes mid-run, evidence collected so far is lost unless the run is resumed with the same `--thread-id` (LangGraph checkpointing is wired in `main.py`).

---

7. Environment and Observability

Environment Variables

Variable	Purpose	Required
`LANGCHAIN_TRACING_V2=true`	Enables LangSmith distributed tracing	No
`LANGCHAIN_API_KEY`	LangSmith authentication	No
`ANTHROPIC_API_KEY`	Claude Haiku for all three judge personas; Claude Haiku as vision fallback	For judges
`GOOGLE_API_KEY`	Gemini 2.5 Flash for VisionInspector diagram analysis	For vision

`RepoInvestigator` and `DocAnalyst` operate entirely without LLM calls — they are deterministic forensic tools. `VisionInspector` requires `GOOGLE_API_KEY` for primary diagram classification and falls back to `ANTHROPIC_API_KEY` (Haiku) if unavailable. The Judicial Layer requires `ANTHROPIC_API_KEY` for all three judge personas; without it, each judge emits a score-1 fallback opinion.

Observability Architecture

```
flowchart LR
  langgraph_invoke["LangGraph\ngraph.invoke()"] -->|LANGCHAIN_TRACING_V2=true| langsmith["LangSmith\nTracing"]
  langsmith --> trace_view["Trace View\n(node timeline,\nstate snapshots,\ntoken usage)"]
  langgraph_invoke -->|stdout| terminal["CLI Summary\n(evidence table)"]
  langgraph_invoke -->|AuditReport| output["ChiefJustice\n(AuditReport in state)"]
```

Each node execution is automatically captured as a LangSmith span, showing the input state, output state delta, and latency. This is essential for debugging the parallel fan-outs — without it, determining which detective or judge emitted which output (or failed silently) requires inspecting raw state diffs.

The two-layer parallel topology (detective fan-out + judicial fan-out) produces a characteristic LangSmith trace shape: two "width" spikes separated by the `EvidenceAggregator` node, with the `ChiefJustice` node appearing as a single leaf after the second fan-in.

Automaton Auditor — built with LangGraph, Pydantic v2, PyMuPDF, and uv.